

# ЛАБОРАТОРНА РОБОТА №3

Тема: Реалізація структури бази даних на фізичному рівні

Мета: на основі логічної моделі даних інформаційної бази системи ROOM\_9 побудувати модель бази даних на фізичному рівні та згенерувати код створення необхідних об'єктів бази даних.

## 1. Обґрунтування вибору системи управління інформаційною базою

Для реалізації інформаційної бази проєкту ROOM\_9 обрано Supabase, який використовує PostgreSQL як основну реляційну СУБД. Такий вибір відповідає структурі предметної області: система містить користувачів, рольові профілі, заявки на бронювання, треки, події, стріми, повідомлення та аналітичні події, між якими існують чіткі зв'язки один-до-багатьох і багато-до-багатьох.

- Реляційна модель. PostgreSQL дає змогу явно описати первинні та зовнішні ключі, каскадне видалення, унікальність та CHECK-обмеження.
- Надійність транзакцій. Заявки на бронювання, статуси та майбутні платежі потребують ACID-семантики.
- Інтеграція з авторизацією. Supabase Auth створює записи в auth.users, а trigger автоматично синхронізує їх із таблицею profiles.
- Безпека на рівні рядків. Row Level Security дозволяє обмежити доступ до профілів, заявок, повідомлень і файлів без використання service\_role key на frontend.
- Файлове сховище. Supabase Storage використовується для audio-файлів у bucket tracks і зображень у bucket images.
- Готовність до деплою. Supabase добре інтегрується з Next.js/Vercel і дає REST/Realtime API поверх PostgreSQL.

## 2. Фізична модель бази даних ROOM\_9

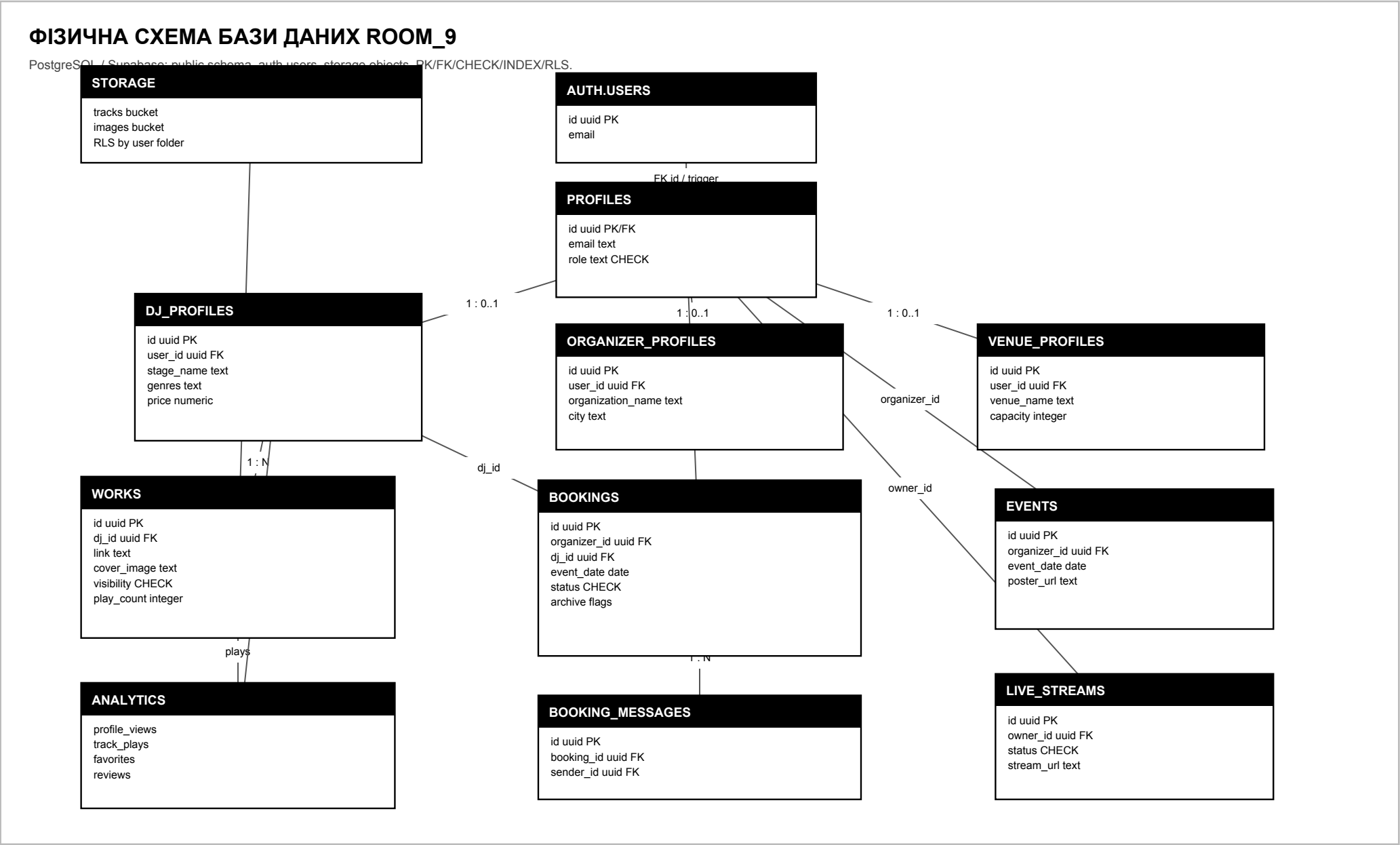
Фізична модель реалізована в PostgreSQL у схемі public. Для ідентифікаторів використовується тип uuid з генерацією через gen\_random\_uuid(), для часу - timestamp with time zone, для вартості бронювання - numeric, для статусів і ролей - text із CHECK-обмеженнями.

| Таблиця            | Призначення                            | Ключі та обмеження                                                         | Основні індекси                                 |
|--------------------|----------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------|
| profiles           | Базові користувачі та ролі.            | PK id; FK auth.users(id); CHECK role in dj/organizer/admin/listener/venue. | -                                               |
| dj_profiles        | Публічні DJ-профілі.                   | PK id; FK user_id; UNIQUE user_id; gen_random_uuid().                      | city, country, genres, is_available, created_at |
| organizer_profiles | Дані організаторів.                    | PK id; FK user_id; UNIQUE user_id.                                         | -                                               |
| venue_profiles     | Фізичні клуби та майданчики.           | PK id; FK user_id; UNIQUE user_id.                                         | city                                            |
| bookings           | Заявки на бронювання DJ.               | PK id; FK organizer_id; FK dj_id; CHECK status.                            | dj_id, organizer_id, status, created_at         |
| booking_messages   | Повідомлення в межах заявки.           | PK id; FK booking_id; FK sender_id.                                        | booking_id, sender_id, created_at               |
| works              | Треки, обкладинки, лірика, статистика. | PK id; FK dj_id; CHECK visibility.                                         | dj_id, visibility, is_deleted, play_count       |
| events             | Події організаторів та venue.          | PK id; FK organizer_id.                                                    | event_date, city                                |
| live_streams       | Сторінка стрімів і архівів сетів.      | PK id; FK owner_id; CHECK status.                                          | status, starts_at                               |
| profile_views      | Аналітика переглядів DJ-профілів.      | PK id; FK dj_id; optional FK viewer_id.                                    | dj_id                                           |
| track_plays        | Аналітика прослуховувань.              | PK id; FK work_id; FK dj_id; optional FK listener_id.                      | dj_id, work_id                                  |
| favorites          | Збережені DJ.                          | PK id; FK user_id; FK dj_id; UNIQUE user_id+dj_id.                         | user_id, dj_id                                  |

| Таблиця  | Призначення                      | Ключі та обмеження                                                 | Основні індекси          |
|----------|----------------------------------|--------------------------------------------------------------------|--------------------------|
| reviews  | Оцінки після бронювань.          | PK id; FK booking_id; reviewer_id; reviewee_id; CHECK rating 1..5. | reviewee_id, reviewer_id |
| payments | Майбутній escrow/payment ledger. | PK id; FK booking_id; payer_id; receiver_id; CHECK status.         | booking_id, status       |

### 3. Структура бази даних у вигляді схеми

Схема нижче показує фізичні таблиці, типи ключів, основні поля та інтеграцію з auth.users і Supabase Storage.



## 4. Коди створення необхідних об'єктів бази даних

Нижче наведено ключові фрагменти SQL-коду. Повний виконуваний файл створення структури БД знаходиться у проєкті за шляхом `supabase/schema.sql`.

### 4.1. Розширення та основні таблиці користувачів/бронювань

```
create extension if not exists pgcrypto;

create table if not exists public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text,
  role text not null check (role in ('dj', 'organizer', 'admin', 'listener', 'venue')),
  created_at timestamp with time zone default now()
);

create table if not exists public.dj_profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  stage_name text,
  bio text,
  country text,
  city text,
  genres text,
  bpm_range text,
  price numeric,
  avatar_url text,
  cover_image_url text,
  profile_theme text,
  soundcloud_url text,
  mixcloud_url text,
  is_available boolean default true,
  created_at timestamp with time zone default now(),
  unique (user_id)
);

create table if not exists public.organizer_profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  organization_name text,
  country text,
  city text,
  contact_email text,
  description text,
  created_at timestamp with time zone default now(),
  unique (user_id)
);

create table if not exists public.bookings (
  id uuid primary key default gen_random_uuid(),
  organizer_id uuid not null references public.profiles(id) on delete cascade,
  dj_id uuid not null references public.dj_profiles(id) on delete cascade,
  event_date date not null,
  venue_name text not null,
  city text not null,
  event_type text not null,
  message text,
  status text default 'pending'
  check (status in ('pending', 'accepted', 'declined', 'cancelled', 'completed', 'paid',
'disputed')),
  archived_by_dj boolean default false,
  archived_by_organizer boolean default false,
  created_at timestamp with time zone default now()
);

create table if not exists public.booking_messages (
  id uuid primary key default gen_random_uuid(),
  booking_id uuid not null references public.bookings(id) on delete cascade,
  sender_id uuid not null references public.profiles(id) on delete cascade,
  message text not null,
  created_at timestamp with time zone default now()
);
```

## 4.2. Таблиці музичного контенту, подій та аналітики

```
create table if not exists public.works (
  id uuid primary key default gen_random_uuid(),
  dj_id uuid not null references public.dj_profiles(id) on delete cascade,
  title text,
  type text default 'track',
  link text,
  description text,
  cover_image text,
  lyrics text,
  genre text,
  bpm text,
  key text,
  visibility text default 'public' check (visibility in ('public', 'private')),
  play_count integer default 0,
  like_count integer default 0,
  is_deleted boolean default false,
  created_at timestamp with time zone default now()
);

create table if not exists public.events (
  id uuid primary key default gen_random_uuid(),
  organizer_id uuid references public.profiles(id) on delete set null,
  title text not null,
  description text,
  venue_name text,
  city text,
  country text,
  event_date date,
  event_type text,
  lineup text,
  poster_url text,
  created_at timestamp with time zone default now()
);

create table if not exists public.live_streams (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid references public.profiles(id) on delete set null,
  title text not null,
  artist_name text not null,
  location text,
  genre text,
  status text default 'upcoming' check (status in ('live', 'upcoming', 'archived')),
  starts_at timestamp with time zone,
  embed_url text,
  stream_url text,
  thumbnail_url text,
  created_at timestamp with time zone default now()
);

create table if not exists public.profile_views (
  id uuid primary key default gen_random_uuid(),
  dj_id uuid not null references public.dj_profiles(id) on delete cascade,
  viewer_id uuid references public.profiles(id) on delete set null,
  created_at timestamp with time zone default now()
);

create table if not exists public.track_plays (
  id uuid primary key default gen_random_uuid(),
  work_id uuid not null references public.works(id) on delete cascade,
  dj_id uuid not null references public.dj_profiles(id) on delete cascade,
  listener_id uuid references public.profiles(id) on delete set null,
  created_at timestamp with time zone default now()
);
```

## 4.3. Підтримувальні та перспективні таблиці V2/V3

```

create table if not exists public.venue_profiles (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  venue_name text,
  country text,
  city text,
  address text,
  capacity integer,
  description text,
  website_url text,
  instagram_url text,
  created_at timestamp with time zone default now(),
  unique (user_id)
);

create table if not exists public.favorites (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  dj_id uuid not null references public.dj_profiles(id) on delete cascade,
  created_at timestamp with time zone default now(),
  unique (user_id, dj_id)
);

create table if not exists public.reviews (
  id uuid primary key default gen_random_uuid(),
  booking_id uuid references public.bookings(id) on delete set null,
  reviewer_id uuid not null references public.profiles(id) on delete cascade,
  reviewee_id uuid not null references public.profiles(id) on delete cascade,
  rating integer not null check (rating between 1 and 5),
  comment text,
  created_at timestamp with time zone default now()
);

create table if not exists public.payments (
  id uuid primary key default gen_random_uuid(),
  booking_id uuid references public.bookings(id) on delete cascade,
  payer_id uuid references public.profiles(id) on delete set null,
  receiver_id uuid references public.profiles(id) on delete set null,
  amount numeric,
  currency text default 'EUR',
  status text check (status in ('pending', 'paid', 'released', 'refunded', 'failed')),
  provider text,
  created_at timestamp with time zone default now()
);

```

## 4.4. Індеси для прискорення пошуку та dashboard-запитів

```
create index if not exists idx_dj_profiles_city on public.dj_profiles(city);
create index if not exists idx_dj_profiles_country on public.dj_profiles(country);
create index if not exists idx_dj_profiles_genres on public.dj_profiles(genres);
create index if not exists idx_dj_profiles_is_available on public.dj_profiles(is_available);
create index if not exists idx_bookings_dj_id on public.bookings(dj_id);
create index if not exists idx_bookings_organizer_id on public.bookings(organizer_id);
create index if not exists idx_bookings_status on public.bookings(status);
create index if not exists idx_works_dj_id on public.works(dj_id);
create index if not exists idx_works_visibility on public.works(visibility);
create index if not exists idx_events_event_date on public.events(event_date);
create index if not exists idx_events_city on public.events(city);
create index if not exists idx_live_streams_status on public.live_streams(status);
create index if not exists idx_profile_views_dj_id on public.profile_views(dj_id);
create index if not exists idx_track_plays_work_id on public.track_plays(work_id);
```

## 4.5. Trigger авторизації та базові RLS-політики

```
create or replace function public.handle_new_user()
returns trigger
language plpgsql
security definer
set search_path = public
as $$
declare
    selected_role text;
begin
    selected_role := coalesce(new.raw_user_meta_data -> 'role', 'organizer');
    if selected_role not in ('dj', 'organizer', 'admin', 'listener', 'venue') then
        selected_role := 'organizer';
    end if;

    insert into public.profiles (id, email, role)
    values (new.id, new.email, selected_role)
    on conflict (id) do update
        set email = excluded.email,
            role = excluded.role;

    return new;
end;
$$;

create trigger on_auth_user_created
after insert on auth.users
for each row execute procedure public.handle_new_user();

alter table public.profiles enable row level security;
alter table public.dj_profiles enable row level security;
alter table public.bookings enable row level security;
alter table public.works enable row level security;

create policy "demo_profiles_update_own"
on public.profiles for update
to authenticated
using (auth.uid() = id)
with check (auth.uid() = id);

create policy "demo_dj_profiles_update_own"
on public.dj_profiles for update
to authenticated
using (auth.uid() = user_id)
with check (auth.uid() = user_id);
```

## 4.6. Storage buckets для аудіо та зображень

```
insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
values (
  'tracks',
  'tracks',
  true,
  52428800,
  array['audio/mpeg', 'audio/mp3', 'audio/wav', 'audio/x-wav']
)
on conflict (id) do update
set public = excluded.public,
  file_size_limit = excluded.file_size_limit,
  allowed_mime_types = excluded.allowed_mime_types;

insert into storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
values (
  'images',
  'images',
  true,
  10485760,
  array['image/jpeg', 'image/png', 'image/webp', 'image/gif']
)
on conflict (id) do update
set public = excluded.public,
  file_size_limit = excluded.file_size_limit,
  allowed_mime_types = excluded.allowed_mime_types;

create policy "demo_tracks_insert_owner_folder"
on storage.objects for insert
to authenticated
with check (
  bucket_id = 'tracks'
  and auth.uid()::text = (storage.foldername(name))[1]
);
```

## 5. Висновок

У результаті лабораторної роботи логічну модель ROOM\_9 було реалізовано на фізичному рівні у PostgreSQL/Supabase. Структура містить типізовані таблиці, первинні та зовнішні ключі, CHECK-обмеження, унікальні обмеження, індекси, trigger для автоматичного створення профілю користувача, RLS-політики та storage buckets для медіафайлів.

Обрана СУБД забезпечує цілісність даних для booking-flow, безпечну роботу з ролями користувачів, можливість масштабування музичного модуля та підготовку до майбутніх функцій V2/V3: рейтингів, обраного, live-stream sessions, ticketing та escrow-платежів.